

Image Restoration and Style-Conditioned Face-to-Sketch Generation with Autoencoders, Mixture-of-Experts and Conditional GANs

Syed Muhammad Faateh Haneef (23i-0688)

GitHub repository: <https://github.com/FaatehHaneef/Image-Processing-Engine>

Demonstration video: <https://www.loom.com/share/b13ed32a8b2e46afb95efe7f5eb7dc9b>

Abstract - This work designs, trains, evaluates and deploys four generative systems. Three restore 128×128 Oxford-IIIT Pet images corrupted at runtime by salt-and-pepper noise, Gaussian blur or occlusion: a universal convolutional denoising autoencoder with a compressed bottleneck, a hard-routed system in which a CNN classifier selects one of three specialist autoencoders or an identity bypass, and a soft mixture-of-experts in which a gate initialized from the classifier blends all branches and is fine-tuned end to end. The fourth is a style-conditioned pix2pix GAN that turns FS2K face photographs into sketches in three styles. All models were tuned with Optuna, tracked with MLflow, exported to ONNX and served by a FastAPI and React application started with Docker Compose. On the official test set, the universal autoencoder raises corrupted-input SSIM from 0.639 to 0.806, the hard-routing classifier is 99.83% accurate, and the soft mixture-of-experts performs best (SSIM 0.845) by learning to blend the original image back in for mild corruptions. The GAN produces visibly distinct styles for the same face.

Index Terms - denoising autoencoder, image restoration, mixture of experts, conditional GAN, pix2pix, Optuna, ONNX, FastAPI.

I. INTRODUCTION

Real photographs suffer from very different degradations: impulse noise, defocus blur and missing regions. A single model that removes all of them must learn one shared representation, while a set of specialists needs a reliable way of choosing which one to use. We study this trade-off with three increasingly flexible restoration systems that share one data pipeline and are evaluated on the same

fixed test inputs, so their results are directly comparable. A fourth task applies paired image-to-image translation to generating face sketches in a chosen artistic style. All four systems are integrated into one web application.

II. RELATED WORK

Denoising autoencoders. Vincent et al. [1] showed that training an autoencoder to reconstruct clean inputs from corrupted ones forces it to learn robust representations. Convolutional encoder-decoders are the standard architecture for image restoration; U-Net-style skip connections [2] improve detail but allow information to bypass the bottleneck, which the assignment rules out for Task 1.

Loss functions for restoration. Zhao et al. [3] showed that L1 loss trains image restoration networks better than L2 and that combining it with the Structural Similarity Index (SSIM) [4] improves perceived quality. We use this combined loss and tune the mixing weight.

Upsampling artifacts. Transposed convolutions can create checkerboard artifacts; nearest-neighbour upsampling followed by a convolution avoids them [5]. All our decoders use this design.

Mixture of experts. Mixture-of-experts models [6] combine several specialized networks through a gating network. Sparsely gated variants [7] route each input to few experts and add load-balancing losses to prevent the gate from using only some experts ("routing collapse") [7], [8]. Our Task 2 is a hard (top-1) router; our Task 3 is a dense, soft mixture with a balance regularizer.

Conditional GANs. GANs [9] train a generator against a discriminator; conditional GANs [10] give both networks side information. pix2pix [11] established the standard paired translation recipe: a U-Net generator, a PatchGAN discriminator that judges local patches, and an adversarial loss combined with an L1 term weighted by $\lambda = 100$. FS2K [12] is the largest public facial-sketch dataset and provides three artist styles, which we use as the condition.

Tools. Optuna [13] provides Bayesian hyperparameter search (TPE) with trial pruning; MLflow records experiments; ONNX and ONNX Runtime allow framework-independent deployment.

III. DATA AND CORRUPTION PIPELINE

A. Oxford-IIIT Pet (Tasks 1-3)

The Oxford-IIIT Pet dataset [14] provides 3,680 official train-validation and 3,669 official test images. The development set was split 80/20 with random seed 42 into 2,944 training and 736 validation images; the split is stored once and reused by all three tasks. Images were converted to RGB (three files are stored as RGBA) and resized directly to 128×128. The official test set was used only for the final evaluation of each model.

B. Corruptions

Training corruptions are generated at runtime inside the data loader: every time an image is loaded, one of four conditions is chosen with equal probability, with a freshly sampled severity (Table I). Corrupted copies are never stored. Each corruption is implemented as a pure function of the image and a small parameter record, so the same code is used for training, for the deterministic manifests and inside the deployed backend.

TABLE I
CORRUPTION DEFINITIONS

Condition	Training (random per load)	Test: low / medium / high
Clean	unchanged	unchanged

Salt-and-pepper	$p \sim U(0.02, 0.15)$; hit pixels black or white (50/50)	$p = 0.03 / 0.08 / 0.15$
Gaussian blur	kernel $\in \{3, 5, 7\}$, $\sigma \sim U(0.5, 2.5)$	$(3, 0.7) / (5, 1.5) / (7, 2.5)$
Occlusion	1-3 black rectangles, 10-35% of the area	1 / 2 / 3 rectangles, ~10 / 20 / 35%

Two implementation details matter. A salt-and-pepper hit replaces the whole pixel (all three channels), and occlusion rectangles are placed without overlap, so the covered area equals the requested area (unit tests check that it is within one percentage point).

Validation and test corruptions are deterministic. The validation manifest gives each of the 736 validation images exactly one condition (184 per class) with severities drawn from the training ranges; the test manifest gives each of the 3,669 test images a clean version plus all three corruptions at all three levels, i.e. 36,690 test inputs. Each entry stores the type, severity, blur settings, rectangle coordinates and its random seed, and regenerating the manifests reproduces them byte for byte.



Fig. 1. Test-manifest corruptions at the three severities and random runtime training samples.

C. FS2K (Task 4)

FS2K provides 1,058 official training and 1,046 official test photo-sketch pairs. The style label is the style field (0, 1, 2), shown as Style 1, 2, 3. A photo photo/photoN/imageXXXX is paired with sketch/sketchN/sketchXXXX. We held out

15% of the official training set as validation, stratified by style with seed 42 (899 training, 159 validation; 54/52/53 per style). The test set is unbalanced (619/381/46 per style), and in the training data the style is tied to the photo source: Styles 1 and 2 only occur with one photo collection, Style 3 only with the others. Photos and sketches are resized to 128×128; sketches are grayscale and are modelled as one channel. Training augmentation resizes both images to 143×143 and applies the same random 128×128 crop and the same horizontal flip to photo and sketch, which keeps the pair aligned.

IV. TASK 1: UNIVERSAL DENOISING AUTOENCODER

A. Architecture

The encoder has four stages, each a stride-2 3×3 convolution followed by a 3×3 convolution with batch normalization and ReLU, reducing 128×128×3 to 8×8 with widths b, 2b, 4b, 8b. A 1×1 convolution compresses the result to an 8×8×c latent tensor, whose size is the tuned bottleneck dimension (64c values). The decoder maps the latent back with a 1×1 convolution and four stages of nearest-neighbour upsampling and two 3×3 convolutions, ending in a sigmoid RGB output. There are no skip connections: the output depends only on the latent tensor, which a unit test verifies. We chose a spatial latent rather than a dense vector because a dense layer from an 8×8×512 feature map would need tens of millions of weights and discards the spatial layout. Dropout is applied to the latent code during training.

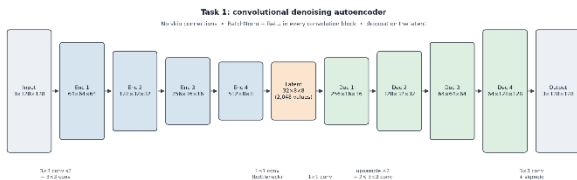


Fig. 2. Task 1 architecture diagram: encoder (128→64→32→16→8, widths b→8b), 1×1 bottleneck to 8×8×c, decoder with upsample+conv, sigmoid output.

B. Loss

For clean image x , corrupted input $\tilde{x}$ and output $\hat{x} = D(E(\tilde{x}))$:

$$L_{\text{UDAE}} = \alpha \cdot L_1(x, \hat{x}) + (1 - \alpha) \cdot (1 - \text{SSIM}(x, \hat{x})) \quad (1)$$

SSIM uses an 11×11 Gaussian window ($\sigma = 1.5$) and is computed in float32. Our implementation was checked against scikit-image on 160 real validation images (maximum difference 1.9×10^{-5}).

C. Training and Hyperparameter Optimization

Training used AdamW (weight decay 10^{-5}), cosine learning-rate decay and mixed precision for the network (losses in float32). Optuna (TPE sampler, seed 42; median pruner) searched the space in Table II with 15-epoch trials. The objective was a fixed validation score, $0.5 \cdot L_1 + 0.5 \cdot (1 - \text{SSIM})$, which deliberately does not contain α : if the training loss itself were the objective, Optuna could lower it by moving α towards the numerically smaller term without improving the images. CUDA out-of-memory errors and non-finite losses were converted into pruned trials.

TABLE II
TASK 1 SEARCH SPACE AND SELECTED CONFIGURATION

Hyperparameter	Search space	Selected
Learning rate	log-uniform [10^{-4} , 3×10^{-3}]	5.8×10^{-4}
Batch size	{16, 32, 64}	16
Bottleneck dimension	{256, 512, 1024, 2048}	2048 (8×8×32)
Encoder base channels	{24, 32, 48, 64}	64
Dropout	[0, 0.3]	0.013
α	[0.5, 0.95]	0.558

The study ran 30 trials: 16 completed, 14 pruned, 0 failed (best trial #24, validation score 0.1495). Dropout was the most important hyperparameter (fANOVA importance 0.75), followed by the learning rate (0.17) and α (0.07). The selected α is lower than the suggested starting value of 0.8, i.e. more weight on SSIM. The final model (7.1 M

parameters) was trained for 100 epochs; the best validation epoch was 96 (score 0.1067, SSIM 0.822, PSNR 26.11 dB).

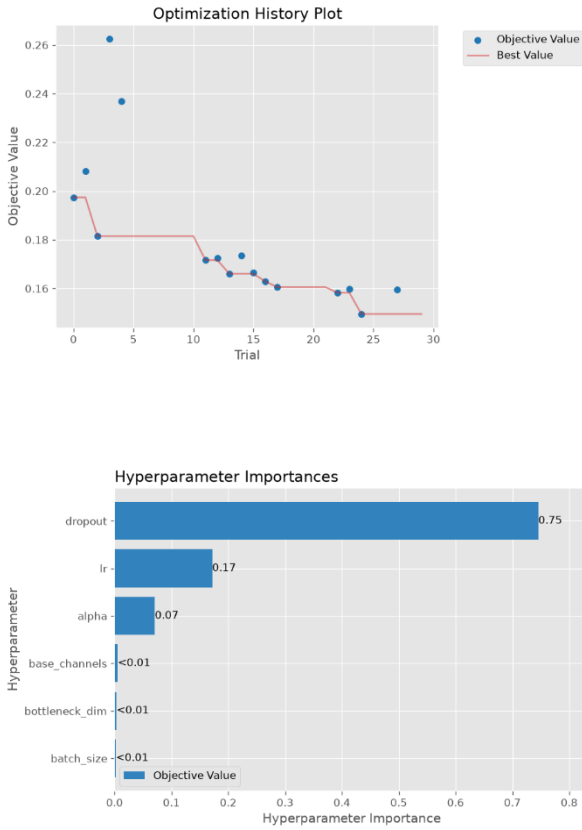


Fig. 3. Optimization history and hyperparameter importance.

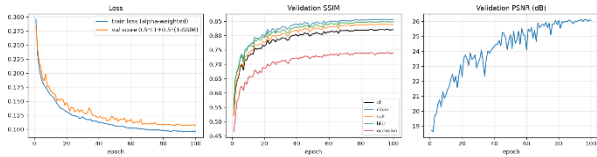


Fig. 4. Training loss, validation score, per-condition validation SSIM and PSNR.

D. Results

TABLE III
TASK 1 TEST RESULTS (MEAN OVER IMAGES;
"INPUT" = CORRUPTED INPUT VS CLEAN TARGET)

Condit ion	Level	PS NR in	PS NR out	SSI M in	SSI M out	L1 out
Clean	—	∞	27. 29	1.0 00	0.8 49	0.0 31

Salt	low	20. 15	27. 27	0.5 88	0.8 48	0.0 31
Salt	medi um	15. 89	27. 21	0.3 23	0.8 44	0.0 31
Salt	high	13. 16	27. 02	0.1 90	0.8 36	0.0 32
Blur	low	34. 06	27. 44	0.9 58	0.8 49	0.0 31
Blur	medi um	28. 06	27. 35	0.8 41	0.8 42	0.0 31
Blur	high	25. 32	26. 29	0.7 31	0.7 89	0.0 34
Occlus ion	low	16. 70	24. 42	0.8 61	0.8 03	0.0 38
Occlus ion	medi um	13. 29	22. 53	0.7 25	0.7 57	0.0 45
Occlus ion	high	10. 78	20. 40	0.5 36	0.6 85	0.0 58
All corrup ted	—	19. 71	25. 55	0.6 39	0.8 06	0.0 37

The autoencoder removes salt-and-pepper noise almost independently of its strength (about 27 dB at every level) and fills occluded regions, raising PSNR by 7.7-9.6 dB. Its outputs, however, are limited to roughly 27 dB and SSIM 0.85 even for clean inputs: everything has to pass through the 2,048-value latent, so fine texture cannot be reproduced. Consequently clean images and low- and medium-blur images come out slightly worse than they went in (low blur: 34.06 $\rightarrow$ 27.44 dB). This ceiling is the main motivation for the identity branch of Tasks 2 and 3.

Task 1: representative test examples (median SSIM of each group)

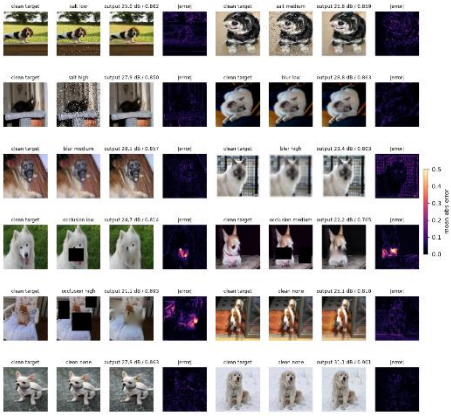


Fig. 5. 12 representative test examples (median-SSIM example of each corruption $\times$ severity group and three clean images): clean target, corrupted input, output, absolute error map.

E. Failure Cases

The four failure cases are the lowest-PSNR test inputs of each condition (Fig. 6). (1, 2) For a cat on a black background, both the clean and the salt-and-pepper versions come out with brightened, greyish borders: the model regresses dark backgrounds towards typical image statistics. (3) Under strong blur, busy high-frequency textures cannot be recovered from the compressed code. (4) When a large occluder hides a distinctive object (a white frisbee), the model fills the hole with plausible background texture, because the information is truly missing.

Task 1: failure cases (lowest output PSNR per condition)

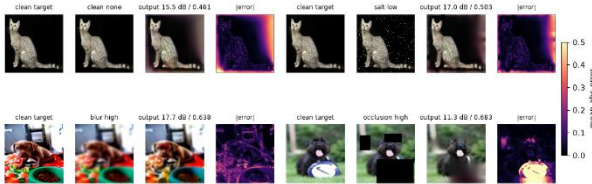


Fig. 6. Four failure cases with error maps.

V. TASK 2: CORRUPTION CLASSIFICATION AND HARD-ROUTED SPECIALISTS

A. Classifier

The classifier has four stages of two 3×3 convolutions with batch normalization and ReLU followed by 2×2 max pooling (widths b , $2b$, $4b$, $8b$), then global average pooling and global max pooling concatenated, dropout

and a linear layer with four outputs. Average pooling summarizes image-wide properties such as missing detail (blur); max pooling keeps the strongest local evidence, such as a few noise pixels or the edge of one rectangle. Training batches are exactly balanced (a quarter of each class in every batch), and the model is trained with cross-entropy. Optuna minimized the validation cross-entropy rather than accuracy because the classifier later initializes the Task 3 gate, whose probabilities are used as mixing weights and should be well calibrated.

TABLE IV
TASK 2 SEARCH SPACES AND SELECTED VALUES

Model	Hyperparameter	Search space	Selected
Classifier	learning rate	$\log [10^{-4}, 3 \times 10^{-3}]$	5.5×10^{-4}
	batch size	{32, 64, 128}	32
	base channels	{16, 24, 32, 48}	48
	dropout	[0, 0.5]	0.32
Specialists	weight decay	$\log [10^{-6}, 10^{-2}]$	2.4×10^{-5}
	learning rate	$\log [10^{-4}, 3 \times 10^{-3}]$	2.1×10^{-3}
	batch size	{16, 32, 64}	16
	bottleneck dimension	{512, 1024, 2048}	2048

	base channels	{24, 32, 40}	40
	α (L1 vs SSIM)	[0.5, 0.95]	0.527

The classifier study ran 25 trials (7 completed, 18 pruned, 0 failed; best validation cross-entropy 0.0135). The final classifier reached 99.73% validation accuracy (best epoch 28; early stopping at 38).

B. Specialists

The three specialists use the Task 1 architecture without dropout and have independent weights. Each is trained and validated only on its own corruption. Following the assignment, one shared Optuna search selected a common configuration: every trial trained all three specialists with the same hyperparameters and was scored by the mean of their validation scores. The three models were reported to the pruner as consecutive steps (salt: epochs 1-10, blur: 11-20, occlusion: 21-30), so trials were always compared at the same specialist and epoch. The study ran 20 trials (10 completed, 10 pruned, 0 failed). Trained for 100 epochs each, the specialists reached validation SSIM 0.844 (salt), 0.848 (blur) and 0.739 (occlusion).

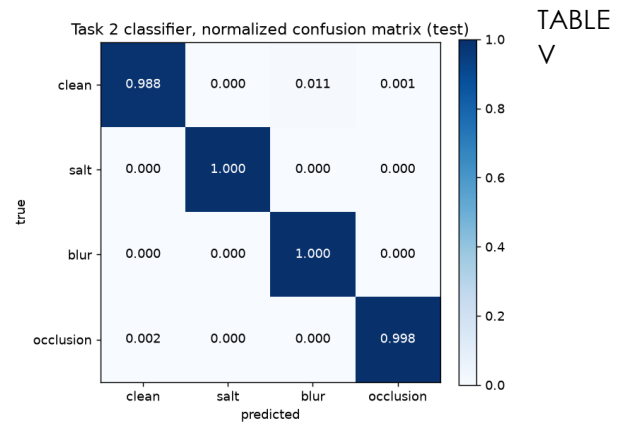
C. Hard Routing

The predicted class $r = \text{argmax}_k p_k$ selects the branch: a clean prediction returns the input unchanged (identity bypass, no expert is run); otherwise the matching specialist restores the image. Two modes were evaluated: oracle routing uses the test-manifest label, predicted routing uses the classifier.

D. Results

On the 36,690 test inputs the classifier reaches 99.83% accuracy with macro precision, recall and F1 of 0.998, 0.997 and 0.997. Per class, F1 is 0.991 (clean), 1.000 (salt), 0.998 (blur) and 0.999 (occlusion). Blur is detected at every level, including the weakest (kernel 3, $\sigma = 0.7$).

Fig. 7. Normalized 4×4 confusion matrix.



RESTORATION SSIM (PSNR IN DB) ON THE TEST SET: TASK 1 VS TASK 2

Condition	Task 1	Task 2 oracle	Task 2 predicted
Clean	0.849 (27.29)	1.000 (∞)	0.999
Salt (all levels)	0.843 (27.17)	0.848 (26.41)	0.848 (26.41)
Blur (all levels)	0.827 (27.03)	0.828 (26.18)	0.828 (26.18)
Occlusion (all levels)	0.748 (22.45)	0.747 (22.18)	0.747 (22.19)
All corrupted	0.806 (25.55)	0.808 (24.92)	0.808 (24.93)
All inputs	0.810	0.827	0.827

Because the classifier is almost always right, oracle and predicted routing are practically identical. The overall gain over Task 1 (SSIM 0.810 $\rightarrow$ 0.827) comes from clean images, which the identity bypass returns unchanged. Per corruption, the specialists match Task 1 in SSIM but are 0.3-0.8 dB lower in PSNR; the specialists are narrower than the Task 1 model (base width 40 vs 64; Section XI) and use a lower α , i.e. more SSIM weight.

E. Failures Caused by Classifier Errors

Only 63 of 36,690 inputs (0.17%) were routed to the wrong branch, in three groups. (1) 41 clean photos were sent to the blur expert. These photos are naturally soft or out of focus; the classifier's decision is understandable, but the blur expert, which has only seen blurred training inputs, shifts the colours of sharp,

saturated images (mean SSIM loss 0.11). (2) Three clean photos of pets on pure black backgrounds were sent to the occlusion expert: large black areas resemble occlusion rectangles. (3) 19 low-occlusion images (one small rectangle) were classified as clean and returned unchanged. Here the error helps: SSIM is 0.085 higher than with the occlusion expert, which degrades the visible 90% of the image more than one small rectangle costs.

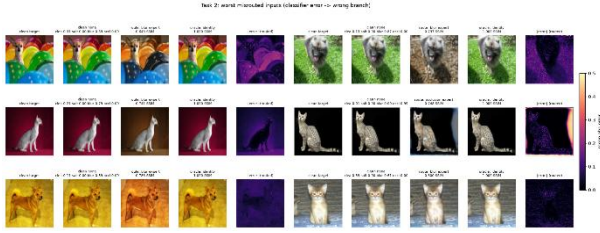


Fig. 8. Worst misrouted inputs: target, input with probabilities, wrongly routed output, oracle output, error map.

VI. TASK 3: SOFT MIXTURE-OF-EXPERTS RESTORATION

A. Model

The gate G has the classifier architecture and produces logits for four branches: identity, salt, blur and occlusion. With temperature T ,

$$w = \text{softmax}(G(\tilde{x})/T), \tilde{x} = w_0 \cdot \tilde{x} + w_1 \cdot A_{\text{salt}}(\tilde{x}) + w_2 \cdot A_{\text{blur}}(\tilde{x}) + w_3 \cdot A_{\text{occ}}(\tilde{x}) \quad (2)$$

All branches run on every input and the weighted sum is differentiable, so the gate and the experts can be trained through the reconstruction error. The gate is initialized from the Task 2 classifier and the experts from the Task 2 specialists; no component starts from random weights.

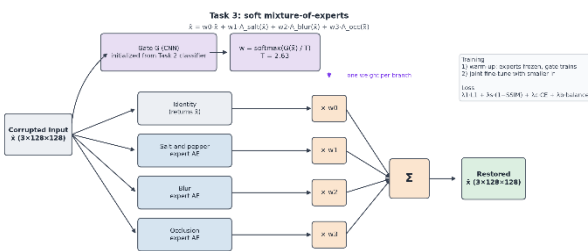


Fig. 9. Task 3 architecture diagram: input $\rightarrow$ gate (softmax with temperature) $\rightarrow$ weights

$w_0..w_3$; identity + three expert autoencoders; weighted sum $\rightarrow$ output.

B. Training

Training has two stages: a warm-up in which the experts are frozen and only the gate is trained (learning rate 5×10^{-4}), followed by joint fine-tuning of all parameters with a smaller, tuned learning rate. During joint training the experts' batch-normalization statistics stay frozen (their weights still train): in the mixture every expert sees every type of input, and updating the running statistics would move each expert's normalization away from the corruption it specializes in. The loss is

$$L_{\text{MoE}} = \lambda_1 \cdot L_1 + \lambda_s \cdot (1 - \text{SSIM}) + \lambda_c \cdot L_{\text{CE}} + \lambda_b \cdot L_{\text{balance}}, L_{\text{balance}} = \sum_{k=0}^3 (\bar{w}_k - 1/4)^2 \quad (3)$$

where $\bar{w}_k$ is the mean weight of branch k in a batch. We kept the suggested balance loss because training batches are exactly balanced (a quarter of each condition), so the target of 1/4 per branch is exactly right. The cross-entropy term is computed on logits/ T , the same distribution that is used as routing weights.

Optuna tuned the joint learning rate (log $[10^{-5}, 3 \times 10^{-4}]$), T ($[0.5, 3]$), λ_c (log $[0.01, 1]$), λ_b (log $[0.001, 0.1]$) and the reconstruction weighting $\lambda_1 = s$, $\lambda_s = 1 - s$ with $s \in [0.5, 0.95]$. Trials used 2 warm-up and 6 joint epochs and were pruned by the median pruner or when routing collapsed on the balanced validation set (a branch with mean weight below 0.05 or above 0.5). The study ran 20 trials (16 completed, 4 pruned, 0 failed). The best trial (#17) used $T = 2.63$, joint learning rate 2.7×10^{-4} , $\lambda_1/\lambda_s = 0.59/0.41$, $\lambda_c = 0.010$ and $\lambda_b = 0.0011$: both regularizers at the lower end of their ranges, i.e. the best mixture is the least constrained one. The final model (3 warm-up and 30 joint epochs) improved the validation score from 0.0860 (untouched Task 2 components) to 0.0702 and validation SSIM from 0.859 to 0.885, while the gate's top-1 agreement with the corruption label fell from 99.7% to 77.5%.

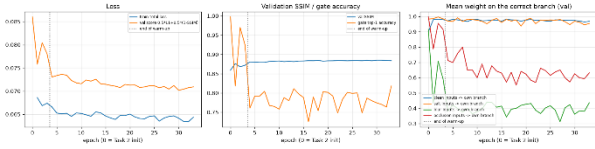


Fig. 10. Training loss, validation SSIM, gate accuracy and mean weight on the correct branch; the end of the warm-up is marked.

C. Results

TABLE VI
TEST SSIM (PSNR IN DB) OF ALL THREE
RESTORATION SYSTEMS

Condition	Level	Task 1	Task 2 (predicted)	Task 3
Clean	—	0.849 (27.29)	0.999 (∞)	0.999 (59.04)
Salt	low / med / high	0.848 / 0.844 / 0.836	0.850 / 0.849 / 0.846	0.864 / 0.854 / 0.849
Blur	low	0.849 (27.44)	0.843 (26.30)	0.950 (33.07)
Blur	medium	0.842 (27.35)	0.841 (26.45)	0.874 (28.75)
Blur	high	0.789 (26.29)	0.800 (25.79)	0.813 (27.06)
Occlusion	low	0.803 (24.42)	0.796 (23.82)	0.886 (23.90)
Occlusion	medium	0.757 (22.53)	0.755 (22.27)	0.810 (22.50)
Occlusion	high	0.685 (20.40)	0.690 (20.49)	0.707 (20.26)

All corrupted	—	0.806 (25.55)	0.808 (24.93)	0.845 (26.18)
---------------	---	---------------	---------------	----------------------

The soft mixture is the best system overall and in every corruption group. The largest gains appear exactly where the autoencoders' quality ceiling hurt most: low blur rises from 26.3 to 33.1 dB, close to the 34.1 dB of the unrestored input, and occlusion SSIM rises from 0.75 to 0.80. For high occlusion, PSNR is slightly below Task 1 (20.26 vs 20.40 dB) while SSIM is higher, because part of the black rectangle is blended back in.

D. Gate Behaviour

TABLE VII
MEAN ROUTING WEIGHTS ON THE TEST SET PER
TRUE CONDITION AND SEVERITY

Condition	Level	Identity	Salt	Blur	Occlusion
Clean	—	0.973	0.000	0.002	0.025
Salt	low / med / high	0.077 / 0.024 / 0.010	0.920 / 0.975 / 0.989	0.000	0.003 / 0.001 / 0.000
Blur	low / med / high	0.788 / 0.550 / 0.464	0.000	0.201 / 0.444 / 0.531	0.011 / 0.005 / 0.005
Occlusion	low / med / high	0.521 / 0.384 / 0.272	0.000	0.000	0.479 / 0.616 / 0.728

The gate learned a severity-dependent policy that nobody programmed: the milder the damage, the more of the original image it keeps (blur: 79/55/46% identity at low/medium/high; occlusion: 52/38/27%). For salt-and-pepper noise it relies almost entirely on the salt expert, since no part of a noisy image is worth keeping. The identity branch

thus acts as a learned, input-dependent shortcut around the bottleneck for mild corruptions. This also explains the drop in top-1 gate accuracy: blending is not classifying, and the reconstruction loss rewards it.

No expert is inactive: each receives at least 0.39 mean weight on its own inputs, and the three corruption experts receive almost no weight on other corruptions (no cross-talk). Our automatic check flags the identity branch as active on corrupted inputs (mean weight 0.34); as Table VI shows, this is beneficial blending rather than collapse.

On the 63 inputs that hard routing sent to the wrong branch, the soft mixture is better in 95% of cases (mean SSIM $0.891 \rightarrow 0.974$), above even Task 2's oracle routing (0.947): clean photos that the classifier called blurred now receive 94% identity weight.

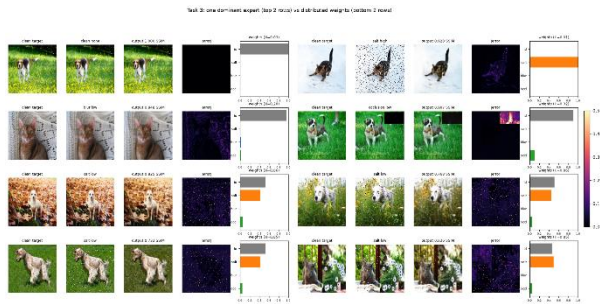
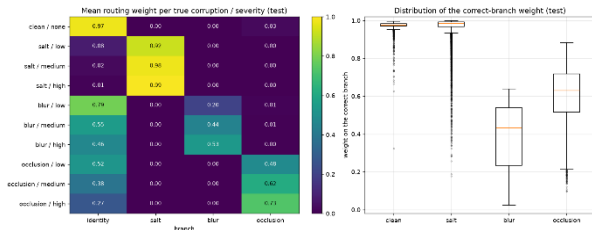


Fig. 11. Mean routing weights per corruption and severity, and the distribution of the weight on the correct branch.

Fig. 12. Inputs with one dominant expert vs inputs with weights spread over several



branches, with the weight bars.

VII. TASK 4: STYLE-CONDITIONED FACE-TO-SKETCH GENERATION

A. Architecture

The generator is a pix2pix U-Net [11]: seven stride-2 4×4 convolution stages (widths b , $2b$, $4b$, $8b$, $8b$, $8b$, $8b$; $128 \rightarrow 1$ pixels) and six transposed-convolution stages with skip connections to the mirrored encoder layers, dropout in the three innermost decoder layers and a sigmoid output with one grayscale channel. Skip connections are appropriate here because photo and sketch share the same spatial layout. The style is a learned embedding of the three FS2K styles. It enters the generator twice, as a constant map concatenated to the photo and again at the 1×1 bottleneck, and it enters the discriminator as a map concatenated to the photo and the (real or generated) sketch. The discriminator is a 70×70 PatchGAN with base width 64, producing a 14×14 grid of real/fake logits for a 128×128 input. Weights are initialized from $N(0, 0.02)$ as in pix2pix.

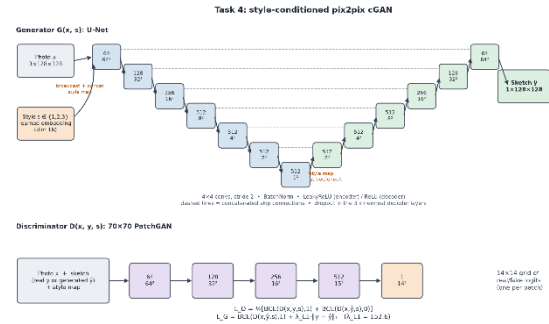


Fig. 13. Task 4 architecture diagram: U-Net generator with style embedding at the input and the bottleneck; PatchGAN discriminator receiving photo, sketch and style map.

B. Objective and Training

With BCE-with-logits losses, the discriminator minimizes $0.5 \cdot [\text{BCE}(D(x, y, s), 1) + \text{BCE}(D(x, G(x, s), s), 0)]$ (logged separately as "D real" and "D fake"), and the generator minimizes

$$L_G = \text{BCE}(D(x, G(x, s), s), 1) + \lambda_{L1} \|y - G(x, s)\|_1 \quad (4)$$

Adam ($\beta_1 = 0.5$) was used with a constant learning rate for the first half of training and linear decay afterwards. The GAN was trained in float32 to avoid any risk of mixed precision destabilizing the adversarial training. Checkpoints were saved every ten epochs, the same six validation photos were rendered

every ten epochs, and trials with non-finite losses or validation L1 above 0.45 were treated as diverged and pruned.

Optuna searched the generator and discriminator learning rates ($\log [5 \times 10^{-5}, 5 \times 10^{-4}]$ each), batch size {4, 8, 16}, generator base channels {32, 48, 64}, dropout [0, 0.5], embedding dimension {4, 8, 16, 32} and λ_{L1} ($\log [10, 200]$), with 20-epoch trials and the fixed validation score $0.5 \cdot L1 + 0.5 \cdot (1 - SSIM)$. The study ran 20 trials (19 completed, 1 pruned, 0 failed). The best trial (#11) used base 64, batch 8, learning rates 3.4×10^{-4} (G and D), dropout 0.33, embedding size 16 and $\lambda_{L1} = 152.6$. Retrained for 150 epochs, the model reached its best validation score at epoch 55 (0.288, SSIM 0.513); later epochs drifted to 0.309. This is typical of GAN training: as the adversarial term pushes towards crisper strokes, pixel metrics, which reward averaged strokes, become slightly worse. The checkpoint was chosen on the validation set.

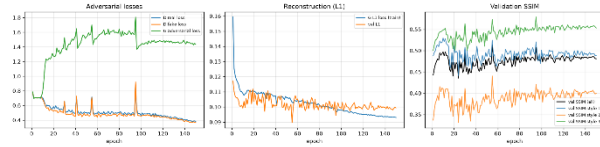


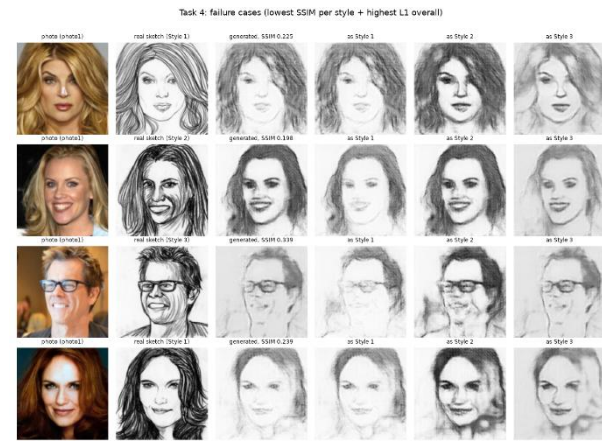
Fig. 14. D real, D fake and G adversarial losses, G L1 loss and validation L1, validation SSIM per style.

C. Results

TABLE VIII
TASK 4 TEST RESULTS (1,046 PAIRS, GENERATED WITH THE TRUE STYLE)

Group	n	L1	SSIM	PSNR (dB)	Style effect
All	1,046	0.099	0.498	16.36	0.110
Style 1	619	0.075	0.542	18.14	0.111
Style 2	381	0.142	0.411	13.10	0.110
Style 3	46	0.061	0.626	19.50	0.094

"Style effect" is the mean absolute



difference between the sketch generated with the true style and the sketches generated with the other two styles for the same photo. A value of 0.11 (11% of the grey range) confirms that the embedding controls the output. Fig. 15 shows the same photo rendered in all three styles: Style 1 produces light, thin lines, Style 2 dark and heavy shading, Style 3 soft grey tones, matching the artists' styles. Style 2 is the hardest (SSIM 0.41) because its dense shading is penalized heavily by pixel metrics whenever strokes are slightly displaced. Style-3 results rest on only 46 test images. Because style and photo source are linked in the training data, per-style numbers partly reflect photo sources (e.g. Style 1: SSIM 0.525 on one source, 0.583 on the other). Sketches are mostly white paper with thin lines, so L1 and SSIM penalize plausible strokes that are a few pixels off; visual inspection of the fixed validation samples complemented the metrics.

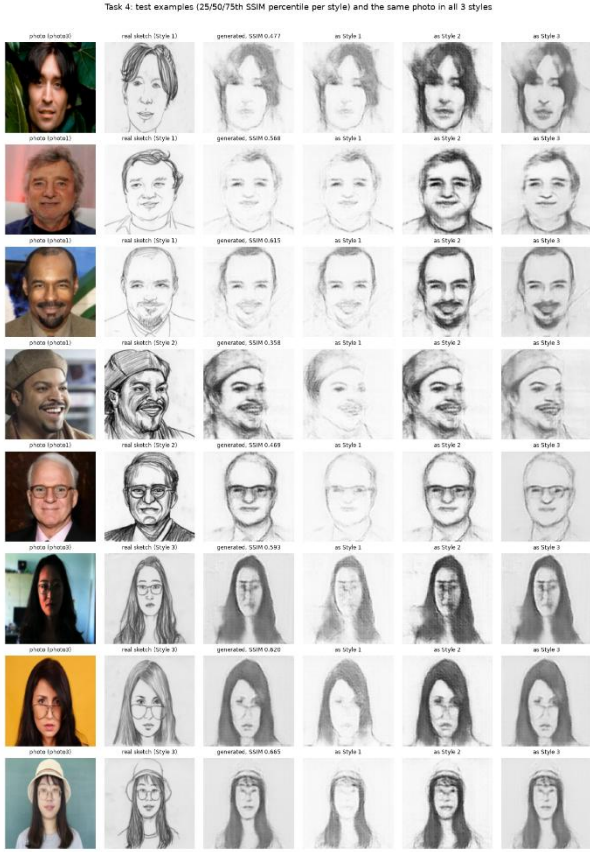
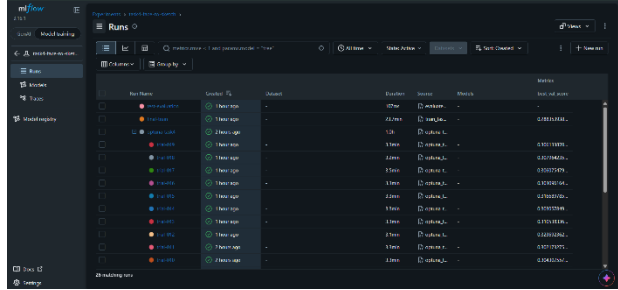


Fig. 15. Test photos, real sketches, generated



sketches and the same photo in all three styles.

Fig. 16. Failure cases (lowest SSIM per style and highest L1).

VIII. HYPERPARAMETER OPTIMIZATION AND EXPERIMENT TRACKING

All studies used persistent SQLite storage, the TPE sampler (seed 42), the median pruner and a fixed validation objective; the test set never entered any objective, early-stopping decision or checkpoint choice. Out-of-memory errors and diverging runs became pruned trials.

TABLE IX
OPTUNA STUDIES

Study	Trials	Completed	Pruned	Failed	Epochs per trial	Best trial
Task 1 universal AE	30	16	14	0	15	#24
Task 2 classifier	25	7	18	0	12	#23
Task 2 specialists	20	10	10	0	3 × 10	#17
Task 3 soft MoE	20	16	4	0	2 + 6	#17
Task 4 cGAN	20	19	1	0	20	#11

MLflow tracked one experiment per task: each study is a parent run with one nested run per trial, and final training and test-evaluation runs log parameters, per-epoch metrics, checkpoints, sample images and result tables.

Fig. 17. MLflow screenshot: the four experiments and one Optuna parent run with its nested trial runs.

IX. ONNX EXPORT AND VERIFICATION

All seven inference models were exported to ONNX in evaluation mode (dropout off, batch-normalization statistics frozen) with a fixed 128×128 input and a dynamic batch dimension. The complete Task 3 pipeline (gate, temperature, softmax, identity branch, three experts and weighted sum) is a single ONNX graph that returns the restored image, the four weights and the logits. The classifier returns both logits and probabilities, and the generator takes a photo and an integer style. Each model was compared with PyTorch on real validation inputs (64 Pet images, 16 per

corrupted. Results show the restored image, an error map when the clean original is known, the classifier probabilities and selected expert (Hard-Routed), the mixture weights and a routing diagram (Soft-MoE), the inference time and a download button. Face-to-Sketch accepts an upload or a webcam capture and one of the three styles.

The FastAPI backend validates uploads (type, size, decodability), applies the training preprocessing, runs the models with ONNX Runtime on the CPU and returns images, routing information and timings through health, sample, corruption, universal, hard-routing, soft-mixture and face-to-sketch endpoints. Single-image inference took roughly 10-40 ms.

C. Deployment

The backend image (Python 3.11 slim with ONNX Runtime; no PyTorch) and the frontend image (a Node build stage followed by nginx, which proxies /api to the backend service) are started with docker compose up --build; the application is then available at <http://localhost:8080>. Models are mounted read-only instead of being copied into images, both services have health checks, and logs are size-limited. The complete procedure (clone, download the large model, start) was tested from a fresh clone of the repository.

XI. LIMITATIONS

Hardware. All training ran on a laptop GPU (NVIDIA RTX 3050 Ti, 4 GB VRAM). This shaped several design decisions: batch sizes were limited (at most 64 for the autoencoders and 16 for the GAN), Optuna trials were short (10-20 epochs) and used pruning, studies were limited to 20-30 trials, and the GAN was trained in float32 rather than with mixed precision.

Model size. In Tasks 1 and 2 the width of the autoencoders was bounded (base channels at most 64 for Task 1 and 40 for the specialists) so that the exported ONNX files, including the single-graph Task 3 model that contains all three specialists, could be distributed directly with the repository. In both studies the

selected configuration was at the upper end of the allowed range.

Bottleneck and resolution. The required compressed bottleneck without skip connections limits Tasks 1 and 2 to about 27 dB and SSIM 0.85, and all models work at the prescribed 128×128 resolution, so restored images look soft when enlarged.

Data. FS2K provides about 300 training pairs per style, its style labels are tied to the photo source in the training data, and only 46 test images belong to Style 3. Per-style results must be read with this in mind.

Metrics. PSNR, SSIM and L1 measure pixel agreement. For sketches in particular they favour averaged strokes over crisp but slightly displaced ones, so they were complemented by visual inspection.

XII. CONCLUSION

Using one shared data pipeline and fixed test inputs, we compared three restoration strategies. A single universal autoencoder with a genuine bottleneck removes heavy corruptions well but cannot exceed its reconstruction ceiling, which makes it degrade clean and mildly corrupted images. Hard routing with a 99.83%-accurate classifier fixes the clean case through an identity bypass but otherwise inherits the specialists' ceiling, and its rare classification errors can be harmful. The soft mixture-of-experts was the best system (corrupted-input SSIM 0.845 vs 0.806 and 0.808): starting from the Task 2 components, it learned without supervision to blend the original image back in proportion to how mild the damage is, and it corrected almost all of hard routing's errors. For face-to-sketch generation, a style-conditioned pix2pix model produces clearly different sketches for the three styles. All models are exported to ONNX, verified against PyTorch and served in a containerized web application. Promising next steps are a larger latent grid or limited, ablated skip connections for the restoration models, and checkpoint selection by visual quality with a lower L1 weight for the GAN.

REFERENCES

[1] P. Vincent, H. Larochelle, Y. Bengio and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proc. ICML*, 2008.

[2] O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015.

[3] H. Zhao, O. Gallo, I. Frosio and J. Kautz, "Loss functions for image restoration with neural networks," *IEEE Trans. Computational Imaging*, vol. 3, no. 1, 2017.

[4] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Processing*, vol. 13, no. 4, 2004.

[5] A. Odena, V. Dumoulin and C. Olah, "Deconvolution and checkerboard artifacts," *Distill*, 2016.

[6] R. A. Jacobs, M. I. Jordan, S. J. Nowlan and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, 1991.

[7] N. Shazeer et al., "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.

[8] W. Fedus, B. Zoph and N. Shazeer, "Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity," *JMLR*, vol. 23, 2022.

[9] I. Goodfellow et al., "Generative adversarial nets," in *Proc. NeurIPS*, 2014.

[10] M. Mirza and S. Osindero, "Conditional generative adversarial nets," arXiv:1411.1784, 2014.

[11] P. Isola, J.-Y. Zhu, T. Zhou and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. CVPR*, 2017.

[12] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin and L. Van Gool, "Facial-sketch synthesis: A new challenge," *Machine Intelligence Research*, vol. 19, 2022.

[13] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD*, 2019.

[14] O. M. Parkhi, A. Vedaldi, A. Zisserman and C. V. Jawahar, "Cats and dogs," in *Proc. CVPR*, 2012.

[15] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[16] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.

[17] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. ICML*, 2015.

APPENDIX A: USE OF AI TOOLS

Tool	Used for	How outputs were tested or corrected
Claude (web chat, Anthropic)	Early planning: understanding the assignment, choosing the overall approach and preparing the working instructions for the coding assistant	Plans were checked against the assignment PDF; decisions were revised during implementation when results or constraints required it
Claude Code (Anthropic, Claude Opus 5.5)	Data pipeline, corruption, model, training, Optuna, evaluation and ONNX code; FastAPI backend; React frontend from the Stitch design; Docker configuration; documentation; first draft of this report	39 unit and API tests (corruption definitions, manifest determinism, SSIM against scikit-image, routing, model properties, upload validation); dry runs before every real training and evaluation; test set evaluated

		once per model; ONNX outputs compared numerically with PyTorch; frontend compared step by step with the Stitch screens and checked with an automated navigation test; full application tested from a fresh clone. All code was reviewed by the student.
Google Stitch	Interface design (layout, colours, components)	Used as the reference for the frontend; decorative placeholder text in the mock-ups was not implemented

Issues found and corrected during development included a version-control rule that would have excluded the data-loading code from the repository, an ONNX exporter crash on Windows, identical images chosen for one example figure, a model-size limit that was reconsidered for Task 4, and test-script errors that produced false failures. Each was detected by a test or by inspection and fixed before results were produced. All reported numbers come from logged runs (MLflow and artifacts/results/).
